ReproZip Docker tutorial

This tutorial creates a small Python program, traces it, builds an `.rpz` bundle, and reruns it through ReproUnzip.

The workflow requires Docker on an x86-64 Linux host. Do not run this tutorial through Docker Desktop's AMD64 emulation on Apple Silicon. The trace failed in that environment during course testing.

1. Check the host

```
uname -s
uname -m
docker version
```

The expected operating system is Linux and the expected architecture is `x86_64`.

2. Get the course repository and build the image

```
git clone https://github.com/gpu004/advance-database.git
cd advance-database

docker build --platform=linux/amd64 -t reprozip:linux-x86 \
  -f reprozip/docker-x86-linux/Dockerfile reprozip/docker-x86-linux
```

3. Create the example on the host

```
mkdir -p "$HOME/reprozip-demo"
cd "$HOME/reprozip-demo"
```

Save this program as `demo.py`:

```
from pathlib import Path
import sys

text = Path(sys.argv[1]).read_text()
Path(sys.argv[2]).write_text(text.upper())
```

Create an input file:

```
printf 'reprozip on nyu linux\n' > input.txt
```

The directory should contain:

```
demo.py
input.txt
```

4. Start the course container

Run this command from `$HOME/reprozip-demo`:

```
docker run --rm -it --platform=linux/amd64 \
  --cap-add=SYS_PTRACE \
  -v "$PWD:/work" -w /work \
  reprozip:linux-x86
```

The current host directory is now `/work` inside the container. Files created below `/work` remain on the host.

Run steps 5 through 10 inside this container.

5. Run the program normally

```
python3 demo.py input.txt output.txt
cat output.txt
```

Expected output:

```
REPROZIP ON NYU LINUX
```

Stop and fix the program if this command fails.

6. Trace the command

```
reprozip trace python3 demo.py input.txt output.txt
```

Confirm that the trace exists:

```
ls -la .reprozip-trace
less .reprozip-trace/config.yml
```

Check the recorded command and review the file list for credentials, private files, and unrelated data.

7. Create the bundle

```
reprozip pack smoke-test.rpz
ls -lh smoke-test.rpz
```

The `.rpz` file remains in `$HOME/reprozip-demo` on the host.

8. Inspect the bundle

```
reprounzip info smoke-test.rpz
reprounzip showfiles smoke-test.rpz
```

The information should include:

- architecture `x86_64`
- the command `python3 demo.py input.txt output.txt`
- compatible unpackers including `directory`
- named input and output files

9. Reproduce it in a fresh directory

Create a separate directory inside the mounted project:

```
mkdir -p check
reprounzip directory setup smoke-test.rpz check/unpacked
reprounzip directory run check/unpacked
```

The command must finish with status 0.

Inspect the reproduced output:

```
cat check/unpacked/root/work/output.txt
```

Expected output:

```
REPROZIP ON NYU LINUX
```

The original output and reproduced output must match.

10. Replace an input

First inspect the file identifiers:

```
reprounzip showfiles smoke-test.rpz
```

The example normally labels the command-line input as `arg2` and the output as `arg3`. Always use the identifiers printed by your own bundle.

Create a replacement input:

```
printf 'second test\n' > new_input.txt
```

Upload and run it:

```
reprounzip directory upload check/unpacked new_input.txt:arg2
reprounzip directory run check/unpacked
```

Download the new output:

```
reprounzip directory download check/unpacked arg3:result.txt
cat result.txt
```

Expected output:

```
SECOND TEST
```

Enter `exit` to leave the container. The trace, bundle, unpacked directory, and result remain in `$HOME/reprozip-demo` on the host.

11. Start the container again later

Return to the project directory and repeat the same command:

```
cd "$HOME/reprozip-demo"

docker run --rm -it --platform=linux/amd64 \
  --cap-add=SYS_PTRACE \
  -v "$PWD:/work" -w /work \
  reprozip:linux-x86
```

The container is disposable. The bind-mounted project files are not.

What to submit for a course project

At minimum, submit:

- one clearly named `.rpz` bundle
- a README with the recorded command
- the expected output
- the language and runtime version
- any assumptions about the input files or host

Before submitting, repeat `reprounzip directory setup` and `reprounzip directory run` in a fresh directory. A successful `reprozip pack` command alone does not prove the bundle can be reproduced.

Verified reference result

This workflow was tested on x86-64 Debian 12 with Python 3.11.14, ReproZip 1.3.2, and ReproUnzip 1.3.2. The trace and reproduced run both produced `REPROZIP ON NYU LINUX`.

Course staff must still confirm that the assigned NYU compute server provides Docker and permits `ptrace`.